

1. Write a program to compute the Covariance, Correlation between a pair of attributes. Extend the program to compute the Covariance Matrix and Correlation Matrix.

```
import numpy as np
import pandas as pd

# Define the dataset
data = {
    'X': [15, 20, 30, 40, 55],
    'Y': [10, 33, 20, 25, 35] }

# Create a DataFrame

df = pd.DataFrame(data)

print("Dataset:")

print(df)

# Calculate and print covariance between X and Y

cov_xy = np.cov(df['X'], df['Y'])[0, 1]

print("\nCovariance between X and Y =", cov_xy)

# Calculate and print correlation between X and Y

corr_xy = np.corrcoef(df['X'], df['Y'])[0, 1]

print("Correlation between X and Y =", corr_xy)

# Calculate and print the covariance matrix
cov_matrix = df.cov()

print("\nCovariance Matrix:")

print(cov_matrix)

# Calculate and print the correlation matrix

corr_matrix = df.corr()

print("\nCorrelation Matrix:")

print(corr_matrix)
```

Output:

Dataset:

	X	Y
0	15	10
1	20	33
2	30	20
3	40	25
4	55	35

Covariance between X and Y = 99.75

Correlation between X and Y = 0.6116093652471665

Covariance Matrix:

	X	Y
X	257.50	99.75
Y	99.75	103.30

Correlation Matrix:

	X	Y
X	1.000000	0.611609
Y	0.611609	1.000000

2. Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, export_text
# Step 1: Create the dataset
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong'],
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No']
}
df = pd.DataFrame(data)
print("The Data's in Table\n")
print(df, "\n")
# Step 2: Encode categorical data
le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])
# Step 3: Split features and target
X = df.drop('PlayTennis', axis=1)
y = df['PlayTennis']
# Step 4: Train Decision Tree using Entropy (ID3)
model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=0
)
model.fit(X, y)
# Step 5: Display the Decision Tree
print("Decision Tree:\n")
print(export_text(model, feature_names=list(X.columns)))
# Step 6: Classify a new sample
new_sample = pd.DataFrame({
    'Outlook': ['Sunny'],
    'Temperature': ['Cool'],
    'Humidity': ['High'],
    'Wind': ['Strong']
})
```

```
# Encode new sample using the same mapping
for column in new_sample.columns:
    encoder = LabelEncoder()
    encoder.fit(data[column])
    new_sample[column] = encoder.transform(new_sample[column])
prediction = model.predict(new_sample)
print("\nPrediction:")
if prediction[0] == 1:
    print("Play Tennis = Yes")
else:
    print("Play Tennis = No")
```

Output:

The Data's in Table

	Outlook	Temperature	Humidity	Wind	PlayTennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No

Decision Tree:

```
|--- Wind <= 0.50
|   |--- class: 0
|--- Wind > 0.50
|   |--- Outlook <= 1.50
|       |--- class: 1
|   |--- Outlook > 1.50
|       |--- class: 0
```

Prediction:

Play Tennis = No

3. Write a program to implement k-Nearest Neighbour algorithm to classify the iris dataset. Print both correct and wrong predictions. Python ML library classes can be used for this problem.

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn import metrics

# Load the Iris dataset
iris = load_iris()

# Create the DataFrame
df = pd.DataFrame(iris.data)

# Convert numeric target (0, 1, 2) into class names
class_names = [
    "Iris-setosa",
    "Iris-versicolor",
    "Iris-virginica" ]

df[4] = [class_names[i] for i in iris.target]
# Save the dataset without column names
df.to_csv("Dataset.csv", index=False, header=False)
# KNN Classification
names = [
    'sepal-length',
    'sepal-width',
    'petal-length',
    'petal-width',
    'Class' ]
# Read the dataset
dataset = pd.read_csv("Dataset.csv", names=names)
# Separate features and target
X = dataset.iloc[:, :-1]
y = dataset.iloc[:, -1]
print(X.head())
# Split the dataset into training and testing data
Xtrain, Xtest, ytrain, ytest = train_test_split(
    X,
    y,
    test_size=0.05,
    random_state=0 )

# Create and train the KNN classifier
classifier = KNeighborsClassifier(n_neighbors=5)
classifier.fit(Xtrain, ytrain)

# Predict the test data
ypred = classifier.predict(Xtest)
```

```

# Display Original Label, Predicted Label and Correct/Wrong
print("\n")
print("%-25s %-25s %-25s" %
      ("Original Label", "Predicted Label", "Correct/Wrong"))

print("-" * 75)

for i, label in enumerate(ytest):
    print("%-25s %-25s" % (label, ypred[i]), end="")

    if label == ypred[i]:
        print("%-25s" % "Correct")
    else:
        print("%-25s" % "Wrong")

print("-" * 75)

# Confusion Matrix
print("\nConfusion Matrix:\n",
      metrics.confusion_matrix(ytest, ypred))

# Classification Report
print("\nClassification Report:\n",
      metrics.classification_report(ytest, ypred))

# Accuracy
print(
    "\nAccuracy of the classifier is %0.2f"
    % metrics.accuracy_score(ytest, ypred)
)

```

Output:

	sepal-length	sepal-width	petal-length	petal-width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Original Label	Predicted Label	Correct/Wrong
Iris-virginica	Iris-virginica	Correct
Iris-versicolor	Iris-versicolor	Correct
Iris-setosa	Iris-setosa	Correct
Iris-virginica	Iris-virginica	Correct
Iris-setosa	Iris-setosa	Correct
Iris-virginica	Iris-virginica	Correct
Iris-setosa	Iris-setosa	Correct
Iris-versicolor	Iris-versicolor	Correct

Confusion Matrix:

```
[[3 0 0]
 [0 2 0]
 [0 0 3]]
```

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	3
Iris-versicolor	1.00	1.00	1.00	2
Iris-virginica	1.00	1.00	1.00	3
accuracy		1.00		8
macro avg	1.00	1.00	1.00	8
weighted avg	1.00	1.00	1.00	8

Accuracy of the classifier is 1.00

4. Write a program to implement feature reduction using Principle Component Analysis

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

target_names = iris.target_names

# Standardize the features (important for PCA)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA to reduce to 2 components
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Print explained variance ratio
print(
    "Explained variance ratio by each principal component:",
    pca.explained_variance_ratio_
)

print("Reduced feature shape:", X_pca.shape)

# Plot the PCA-reduced data
plt.figure(figsize=(8, 6))

colors = ['red', 'green', 'blue']

for i, target_name in enumerate(target_names):
    plt.scatter(
        X_pca[y == i, 0],
        X_pca[y == i, 1],
        label=target_name,
        color=colors[i],
        edgecolor='k'    )
```

```
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')

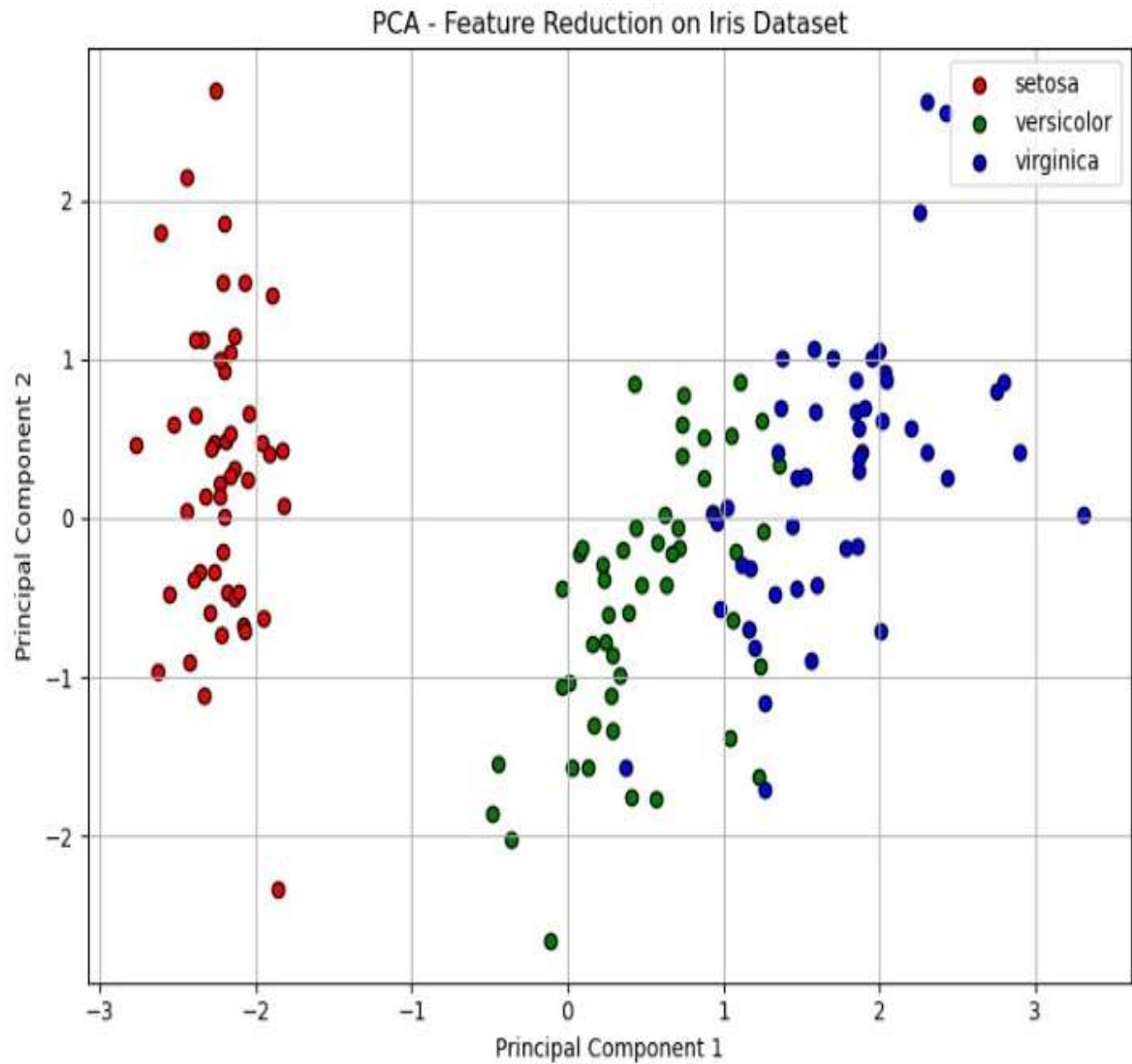
plt.title('PCA - Feature Reduction on Iris Dataset')

plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Output:

Explained variance ratio by each principal component: [0.72962445 0.22850762]

Reduced feature shape: (150, 2)



5. Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering a few test data sets

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Load the Iris dataset from CSV
data = pd.read_csv(
    "Dataset.csv",
    header=None,
    names=[
        "sepal_length",
        "sepal_width",
        "petal_length",
        "petal_width",
        "label"
    ]
)

# Display the first 5 rows
print("Iris Dataset:")
print(data.head())

# Separate features (X) and target/label (y)
X = data.drop("label", axis=1)
y = data["label"]

# Split the dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.05,
    random_state=0
)

# Create the Gaussian Naive Bayes classifier
model = GaussianNB()

# Train the model
model.fit(X_train, y_train)

# Predict the test data
y_pred = model.predict(X_test)

# Display prediction results
print("\n" + "=" * 65)
print("      NAIVE BAYES CLASSIFICATION")
```

```

print("=" * 65)

print(
    f'{'S.No':<6}'
    f'{'Actual Label':<22}'
    f'{'Predicted Label':<22}'
    f'{'Result'}'
)

print("-" * 65)

for i in range(len(y_test)):
    actual = y_test.iloc[i]
    predicted = y_pred[i]

    result = "Correct" if actual == predicted else "Wrong"

    print(
        f'{'i + 1':<6}'
        f'{'actual':<22}'
        f'{'predicted':<22}'
        f'{'result'}'
    )

print("=" * 65)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print(
    "\nAccuracy of Naive Bayes Classifier:",
    accuracy
)

# Confusion Matrix
print("\n--- Confusion Matrix ---")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\n--- Classification Report ---")
print(classification_report(y_test, y_pred))

```

Output:

Iris Dataset:

	sepal_length	sepal_width	petal_length	petal_width	label
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

NAIVE BAYES CLASSIFICATION

S.No	Actual Label	Predicted Label	Result
1	Iris-virginica	Iris-virginica	Correct
2	Iris-versicolor	Iris-versicolor	Correct
3	Iris-setosa	Iris-setosa	Correct
4	Iris-virginica	Iris-virginica	Correct
5	Iris-setosa	Iris-setosa	Correct
6	Iris-virginica	Iris-virginica	Correct
7	Iris-setosa	Iris-setosa	Correct
8	Iris-versicolor	Iris-versicolor	Correct

Accuracy of Naive Bayes Classifier: 1.0

--- Confusion Matrix ---

```
[[3 0 0]
 [0 2 0]
 [0 0 3]]
```

--- Classification Report ---

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	3
Iris-versicolor	1.00	1.00	1.00	2
Iris-virginica	1.00	1.00	1.00	3
accuracy		1.00		8
macro avg	1.00	1.00	1.00	8
weighted avg	1.00	1.00	1.00	8

6. Given a dataset for classification task. Write a program to implement Support Vector Machine and estimate its test performance.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

# Load dataset from CSV
data = pd.read_csv(
    "Dataset.csv",
    header=None,
    names=[
        "sepal_length",
        "sepal_width",
        "petal_length",
        "petal_width",
        "label"
    ]
)

print("Iris Dataset:\n")
print(data.head())

# Separate features and labels
X = data.drop("label", axis=1)
y = data["label"]

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.07,
    random_state=0
)

# Create SVM model
model = SVC(kernel="linear")

# Train the model
model.fit(X_train, y_train)

# Predict on test set
y_pred = model.predict(X_test)

# Display Actual vs Predicted Output
print("\n" + "=" * 65)
print("          SVM CLASSIFICATION")
print("=" * 65)
```

```

print(
    f"{'S.No':<6}"
    f"{'Actual Label':<22}"
    f"{'Predicted Label':<22}"
    f"{'Result'}"
)

print("-" * 65)

for i in range(len(y_test)):
    actual = y_test.iloc[i]
    predicted = y_pred[i]

    result = "Correct" if actual == predicted else "Wrong"

    print(
        f"{i + 1:<6}"
        f"{actual:<22}"
        f"{predicted:<22}"
        f"{result}"
    )

print("=" * 65)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

Output:

Iris Dataset:

	sepal_length	sepal_width	petal_length	petal_width	label
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

SVM CLASSIFICATION

S.No	Actual Label	Predicted Label	Result
1	Iris-virginica	Iris-virginica	Correct
2	Iris-versicolor	Iris-versicolor	Correct
3	Iris-setosa	Iris-setosa	Correct
4	Iris-virginica	Iris-virginica	Correct
5	Iris-setosa	Iris-setosa	Correct
6	Iris-virginica	Iris-virginica	Correct
7	Iris-setosa	Iris-setosa	Correct
8	Iris-versicolor	Iris-versicolor	Correct
9	Iris-versicolor	Iris-versicolor	Correct
10	Iris-versicolor	Iris-versicolor	Correct
11	Iris-virginica	Iris-virginica	Correct

Accuracy: 1.0

Confusion Matrix:

```
[[3 0 0]
 [0 4 0]
 [0 0 4]]
```

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	3
Iris-versicolor	1.00	1.00	1.00	4
Iris-virginica	1.00	1.00	1.00	4
accuracy		1.00		11
macro avg	1.00	1.00	1.00	11
weighted avg	1.00	1.00	1.00	11

7. Build an Artificial Neural Network by implementing the Back propagation algorithm and test the same using appropriate data sets.

```
import numpy as np

# Input dataset
X = np.array([ [0, 0], [0, 1], [1, 0], [1, 1] ], dtype=float)

# Target output - XOR
y = np.array([ [0], [1], [1], [0] ], dtype=float)

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Network structure
input_neurons = 2
hidden_neurons = 3
output_neurons = 1

# Learning parameters
learning_rate = 0.5
epochs = 10000
# Initialize weights and biases
np.random.seed(42)

W1 = np.random.uniform(
    -1, 1,
    (input_neurons, hidden_neurons) )

b1 = np.zeros((1, hidden_neurons))

W2 = np.random.uniform(
    -1, 1,
    (hidden_neurons, output_neurons) )

b2 = np.zeros((1, output_neurons))

# Training
for epoch in range(epochs):

    # ----- Forward Propagation -----
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)

    output_input = np.dot(hidden_output, W2) + b2
    output = sigmoid(output_input)
```

```

# ----- Backpropagation -----
error = y - output

output_gradient = (
    error * sigmoid_derivative(output) )

hidden_error = np.dot(
    output_gradient,
    W2.T )
hidden_gradient = (
    hidden_error * sigmoid_derivative(hidden_output) )

# ----- Update weights and biases -----
W2 += (
    np.dot(hidden_output.T, output_gradient)
    * learning_rate )

b2 += (
    np.sum(
        output_gradient,
        axis=0,
        keepdims=True
    ) * learning_rate )

W1 += (
    np.dot(X.T, hidden_gradient)
    * learning_rate )

b1 += (
    np.sum(
        hidden_gradient,
        axis=0,
        keepdims=True
    ) * learning_rate )

# Display results
print("Input:")
print(X.astype(int))

print("\nActual Output:")
print(y.astype(int))

print("\nPredicted Output:")
print(output)

print("\nPredicted Class:")
print((output >= 0.5).astype(int))

```

Output:

Input:

```
[[0 0]
 [0 1]
 [1 0]
 [1 1]]
```

Actual Output:

```
[[0]
 [1]
 [1]
 [0]]
```

Predicted Output:

```
[[0.01562875]
 [0.98286529]
 [0.98432101]
 [0.01345436]]
```

Predicted Class:

```
[[0]
 [1]
 [1]
 [0]]
```

8. Write a program to implement K-means clustering algorithm. Select your own data set to test the program. Demonstrate the nature of output

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Dataset
X = np.array([ [1, 35], [2, 40], [2, 45], [3, 50], [7, 75], [8, 80], [9, 85], [10, 90] ])

# Different values of K
k_values = [2, 3, 4]

# Create a figure
plt.figure(figsize=(15, 4))

for i, k in enumerate(k_values):

    # Create K-Means model
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10 )

    # Perform clustering
    labels = model.fit_predict(X)

    # Display cluster assignments
    print("\nK =", k)
    print("Cluster Labels:", labels)

    # Display cluster centroids
    print("Cluster Centroids:")
    print(model.cluster_centers_)

    # Create subplot
    plt.subplot(1, 3, i + 1)

    # Plot data points
    plt.scatter(
        X[:, 0],
        X[:, 1],
        c=labels,
        s=100
    )
```

```
# Plot centroids
plt.scatter(
    model.cluster_centers_[:, 0],
    model.cluster_centers_[:, 1],
    marker='X',
    s=200 )

# Add title and labels
plt.title("K-Means Clustering (K = " + str(k) + ")")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.grid(True)

plt.tight_layout()
plt.show()
```

Output:

K = 2

Cluster Labels: [0 0 0 0 1 1 1 1]

Cluster Centroids:

[[2. 42.5]

[8.5 82.5]]

K = 3

Cluster Labels: [1 1 1 1 2 2 0 0]

Cluster Centroids:

[[9.5 87.5]

[2. 42.5]

[7.5 77.5]]

K = 4

Cluster Labels: [2 0 0 0 3 3 1 1]

Cluster Centroids:

[[2.33333333 45.]

[9.5 87.5]

[1. 35.]

[7.5 77.5]]

